

Pythonda tarmoq dasturlash

O'quv Rejasi

I. Tarmoq Arxitekturasini va Soketlar Anatomiyasi

- **1.1. OSI modeli vs TCP/IP:** Ma'lumotning "bo'laklanishi" va "paketlanishi" (Encapsulation).
- **1.2. Soket tushunchasi:** Fayl deskriptorlari, IP-manzillar (IPv4/IPv6) va Portlar klassifikatsiyasi.
- **1.3. TCP Protokoli:** Uch bosqichli "qo'l berib ko'rishish" (3 – *way handshake*) va ulanishni uzish (*FIN/ACK*) mexanizmlari.
- **1.4. Kiberxavfsizlik:** Paketlarni ushlash (*Sniffing*) va "Port Scanning" metodlari.

II. Klient-Server Modelining Dasturiy Realizatsiyasi

- **2.1. Server hayotiy sikli:** `socket()`, `bind()`, `listen()`, `accept()` funksiyalarining operatsion tizim darajasidagi vazifasi.
- **2.2. Klient hayotiy sikli:** `connect()` va xato yuz berganda qayta ulanish algoritmi.
- **2.3. Ma'lumot almashish:** `send()`, `recv()`, `sendall()` farqlari, baytlarni kodlash (*UTF – 8*) va "Buffer Size"ni to'g'ri tanlash.
- **2.4. Xavfsizlik:** Ma'lumotlar butunligini tekshirish (*Checksum/Hashing*).

III. Murakkab Tarmoq Ssenariylari va Xatoliklar Bilan Ishlash

- **3.1. Blocking vs Non-blocking Sockets:** Dastur qotib qolishining oldini olish.
- **3.2. Timeouts va Reconnect:** Tarmoq uzilganda tizimning barqarorligini ta'minlash.
- **3.3. JSON Data Exchange:** Strukturalangan ma'lumotlarni (lug'atlar, ro'yxatlar) tarmoq orqali uzatish.
- **3.4. Kiberxavfsizlik:** "Denial of Service" (DoS) hujumlarini tahlil qilish va himoyalash.

IV. Ko'p Oqimli (Multi-threading) va Asinxron Tizimlar

- **4.1. Threading moduli:** Bir vaqtning o'zida bir nechta mijozni qabul qilish.
- **4.2. Race Conditions:** Umumiy ma'lumotlar (masalan, chat tarixi) bilan ishlashda `Lock` mexanizmi.
- **4.3. Asyncio kutubxonasi:** Zamonaviy yuqori yuklamali (High-load) tizimlar asoslari.

V. Grafik Interfeys (PyQt5) va Chat Loyihasi

- **5.1. PyQt5 GUI dizayni:** Chat oynasi, Login formasi va Signallar (*Signals & Slots*).
- **5.2. Network-Worker Pattern:** Tarmoq kodini GUI oqimidan ajratish (dastur "javob bermay qoldi" holatiga tushmasligi uchun).
- **5.3. Real-time Chat:** Xabarlarini barcha foydalanuvchilarga tarqatish (*Broadcasting*).

VI. Kiberxavfsizlik: Ma'lumotlarni Shifrlash (End-to-End)

- **6.1. Simmetrik shifrlash:** AES algoritmi yordamida xabarlarini himoyalash.

- **6.2. SSL/TLS asoslari:** Sertifikatlar yordamida ulanishni xavfsiz qilish.

VII. Deployment: Loyihani Mustaqil Faylga Aylantirish

- **7.1. PyInstaller:** .exe fayl yaratish va kutubxonalarni paketlash.
 - **7.2. Cross-platform Testing:** Windows va Linux uchun tayyorlash.
-

In []:

I. Tarmoq Arxitekturası va Soketlar Anatomiyasi

1.1. OSI modeli vs TCP/IP: Encapsulation (Ma'lumotning paketlanishi)

Tarmoq orqali ma'lumot uzatish jarayoni **Encapsulation** (qobiqqa o'rash) deb ataladi. Tasavvur qiling, siz xat yuboryapsiz: xatni konvertga solasiz, ustiga manzil yozasiz va markasini yopishtirasiz. Tarmoqda ham xuddi shunday.

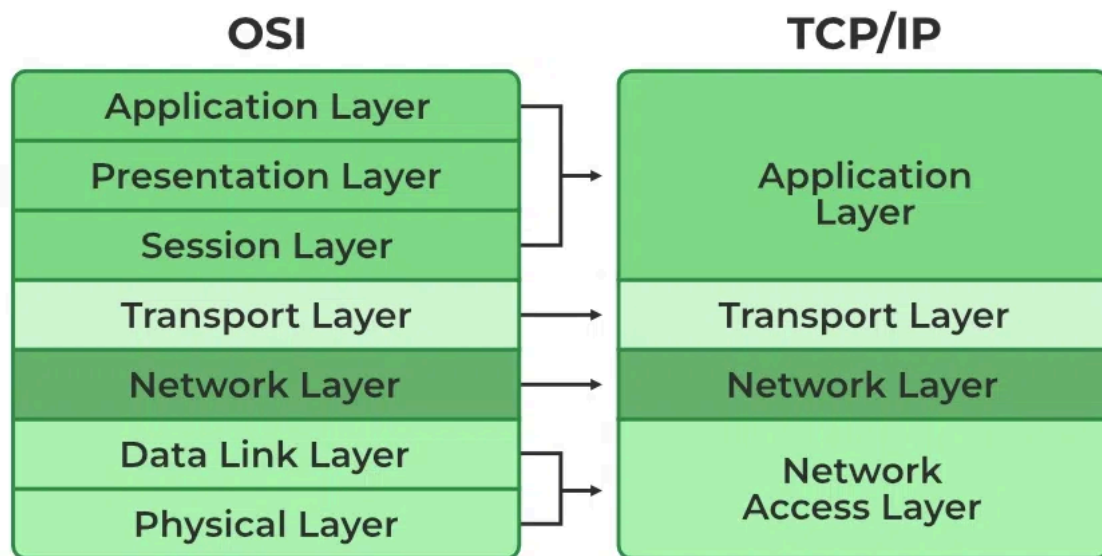
- **OSI (Open Systems Interconnection) modeli:** 7 qatlamdan iborat nazariy model.
- **TCP/IP modeli:** 4 qatlamdan iborat amaliy model (internet shu asosda ishlaydi).

Encapsulation jarayoni:

1. **Application (Ilova):** Sizning dasturingiz (masalan, "Salom" xabari).
2. **Transport (TCP/UDP):** Xabarga **Port** raqamlari qo'shiladi. Ma'lumot **Segment** deb ataladi.
3. **Internet (IP):** Segmentga **IP-manzillar** qo'shiladi. Endi u **Paket** deb ataladi.
4. **Network Access:** Paketga **MAC-manzil** qo'shiladi. Endi u **Freym** (Frame) deb ataladi va bitlar ko'rinishida sim orqali ketadi.

TCP/IP Model Vs. OSI Model

OSI Reference Model	TCP/IP
Application Layer	Application Layer
Presentation Layer	
Session Layer	
Transport Layer	Transport Layer
Network Layer	Internet Layer
Data Link Layer	Link Layer
Physical Layer	
	InstrumentationTools.com



1.2. Soket tushunchasi va Identifikatsiya

Soket — bu dastur va tarmoq o'rtasidagi interfeys. Operatsion tizim nuqtai nazaridan soket bu **Fayl deskriptori** (File Descriptor) hisoblanadi. Ya'ni, Python-da fayl bilan qanday ishlangiz (ochish, yozish, yopish), soket bilan ham shunday ishlaysiz.

IP-manzillar (Manzil)

- **IPv4:** 32-bitli manzil (masalan, 192.168.1.1). Maksimal 2^{32} ta manzil bor.
- **IPv6:** 128-bitli manzil (masalan, 2001:0db8:85a3...). Manzillar yetishmovchiligi muammosini hal qiladi.

Portlar klassifikatsiyasi (Eshiklar)

Port — bu kompyuter ichidagi qaysi dasturga ulanishni belgilovchi raqam (0 dan 65535 gacha):

- **System Ports (0-1023):** Standart xizmatlar (80: HTTP, 443: HTTPS, 22: SSH).
- **User/Registered Ports (1024-49151):** Maxsus dasturlar (masalan, 3306: MySQL).
- **Dynamic/Private Ports (49152-65535):** Vaqtinchalik ulanishlar uchun.

Soketni (Socket) tushunish uchun uni dasturlash va apparat ta'minoti (hardware) o'rtasidagi **"ko'priq"** yoki **"oxirgi nuqta"** deb tasavvur qilish kerak.

Operatsion tizim (Windows, Linux, macOS) uchun socket xuddi **fayl** kabi ko'rinadi. Siz faylni ochasiz, unga ma'lumot yozasiz (`write`) va undan ma'lumot o'qiyasiz (`read`). Tarmoqda ham xuddi shunday: socketga ma'lumot yozasiz, u tarmoq orqali ikkinchi tomonga yetib boradi.

Soketning tarkibiy qismlari (Manzil) Bitta socketni tarmoqda tanib olish uchun 5 ta element kerak (bunga **5-tuple** deyiladi):

1. **Protokol:** (TCP yoki UDP).
2. **Lokal IP-manzil:** Ma'lumot chiqayotgan kompyuter manzili.
3. **Lokal Port:** Ma'lumot chiqayotgan dasturning "eshigi".
4. **Masofaviy IP-manzil:** Ma'lumot boradigan kompyuter manzili.
5. **Masofaviy Port:** Ma'lumot boradigan dasturning "eshigi".

Soket turlari (Protokolga ko'ra) Eng ko'p qo'llaniladigan ikki tur:

- **Stream Sockets (SOCK_STREAM / TCP):**
 - **Xususiyati:** Ishonchli. Ma'lumotlar xuddi oqim (stream) kabi tartib bilan boradi. Agar yo'lda paket yo'qolsa, TCP uni qaytadan yuboradi.
 - **Misol:** HTTP (veb-saytlar), Email, Fayl yuklash.
- **Datagram Sockets (SOCK_DGRAM / UDP):**
 - **Xususiyati:** Tezkor, lekin kafolatsiz. Ma'lumot bo'laklari (paketlar) tartibsiz borishi yoki yo'qolishi mumkin.
 - **Misol:** Video qo'ng'iroqlar, Onlayn o'yinlar (ping kam bo'lishi uchun).

Fayl Deskriptori sifatida Soket Siz Python-da `socket.socket()` funksiyasini chaqirganingizda, operatsion tizim sizga bitta butun son (masalan, 3 yoki 540) qaytaradi. Bu son **File Descriptor** deyiladi.

- Operatsion tizim ichida bitta jadval bor.
- O'sha jadvalda 3 -raqamli socket qaysi IP va qaysi Portga bog'langani yozilgan bo'ladi.
- Siz `send()` buyrug'ini berganingizda, OS o'sha jadvalga qarab ma'lumotni qayerga haydashni bilib oladi.

Soketning hayotiy sikli (Server misolida) Server soketi doimo quyidagi bosqichlardan o'tadi:

1. **Creation (Yaratish):** "Menga aloqa vositasi kerak".
2. **Binding (Bog'lash):** "Men 8080-portda va shu IP-da ishlayman".
3. **Listening (Eshitish):** "Kimdir ulanishini kutib turibman".

Kiberxavfsizlik nuqtai nazaridan socketlar

Soketlarni tushunish xavfsizlik uchun juda muhim:

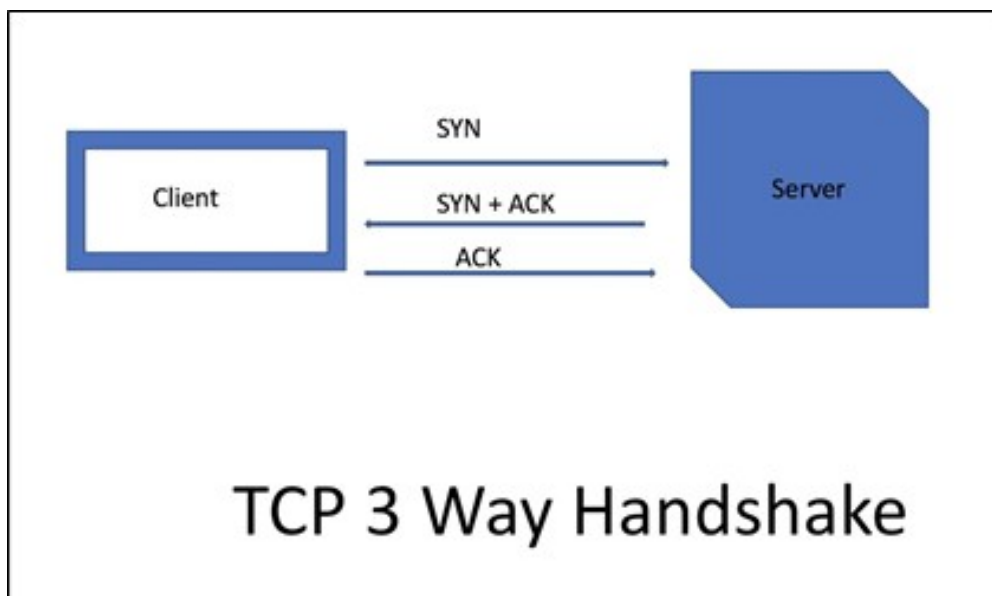
- **Port Exhaustion:** Agar siz ulanishlarni (socketlarni) to'g'ri yopmasangiz (`.close()`), tizimda bo'sh portlar qolmaydi va yangi ulanishlarni qabul qilolmay qoladi. Bu "Denial of Service" (DoS) holatiga sabab bo'ladi.
 - **Raw Sockets:** Ba'zi maxsus socketlar paketning hamma qismini (hatto IP-headerini ham) qo'lda yozishga imkon beradi. Buzg'unchilar bundan **IP Spoofing** (o'zini boshqa IP qilib ko'rsatish) uchun foydalanishadi.
 - **Socket Hijacking:** Ishlab turgan socket ulanishini o'g'irlash orqali ma'lumotlarni qo'lga kiritish mumkin.
-

1.3. TCP Protokoli: Aloqa o'rnatish va uzish

TCP — bu ulanishga asoslangan (**connection-oriented**) protokol. Ma'lumot uzatishdan oldin ikki tomon "kelishib" oladi.

Uch bosqichli salomlashish (3-way Handshake)

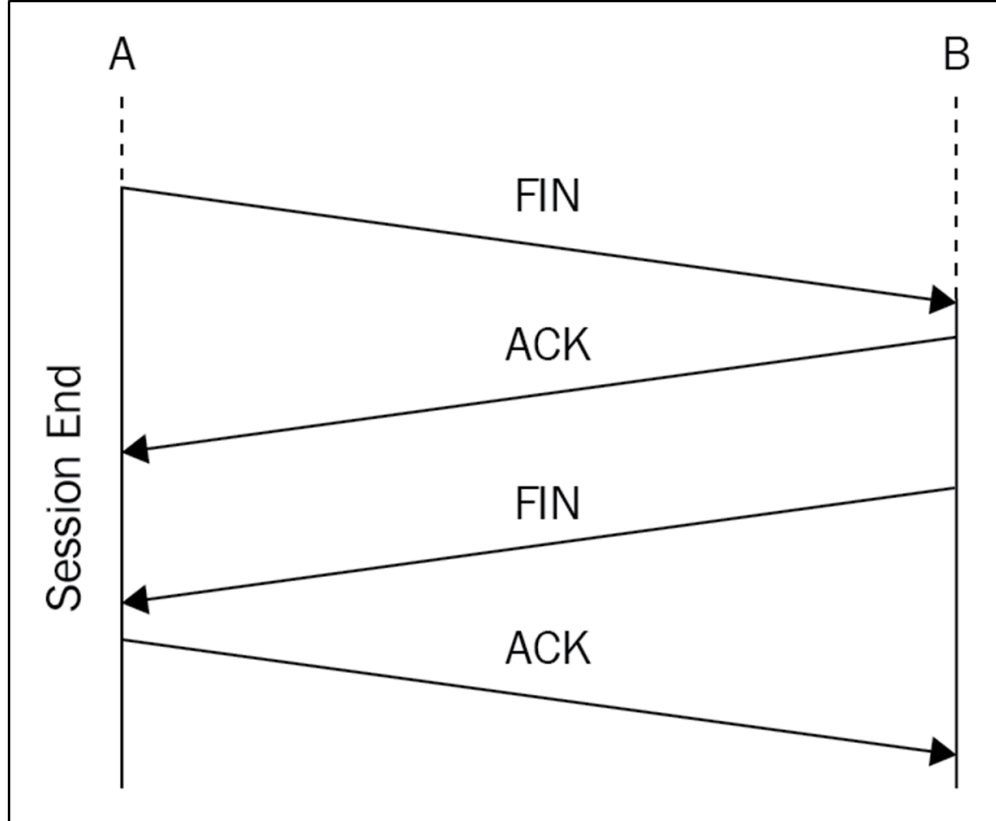
1. **SYN (Synchronize):** Klient serverga ulanish so'rovini yuboradi.
2. **SYN-ACK:** Server so'rovni qabul qiladi va o'zining tayyorligini bildiradi.
3. **ACK (Acknowledge):** Klient ulanish o'rnatilganini tasdiqlaydi.



Ulanishni uzish (4-way Termination)

TCP aloqani uzishda har bir tomon mustaqil ravishda o'z qismini yopishi kerak:

1. Klient **FIN** yuboradi (Men boshqa ma'lumot yubormayman).
2. Server **ACK** yuboradi (Tushundim).
3. Server ham o'z ishini tugatgach, **FIN** yuboradi.
4. Klient **ACK** yuboradi va aloqa butunlay yopiladi.



1.4. Kiberxavfsizlik asoslari

Tarmoq dasturchisi sifatida siz quyidagi xavf va metodlarni bilishingiz shart:

- **Sniffing (Paketlarni ushlash):** Tarmoq trafigini kuzatish. Agar ma'lumot shifrlanmagan bo'lsa (HTTP, FTP kabi), buzg'unchi Wireshark kabi vositalar bilan foydalanuvchi parollarini o'qib olishi mumkin.
 - *Himoya:* SSL/TLS shifrlashdan foydalanish.
- **Port Scanning:** Buzg'unchi nishon kompyuterga turli portlar bo'yicha SYN paketlarini yuborib ko'radi. Agar javob kelsa, demak u yerda qandaydir dastur ishlayapti va bu dasturda zaiflik bo'lishi mumkin.
 - *Himoya:* Firewall (brandmauer) orqali keraksiz portlarni yopish va rate limiting (ulanishlar sonini cheklash) o'rnatish.

In []:

II. Klient-Server Modelining Dasturiy Realizatsiyasi

Ushbu bo'limda biz Python-ning `socket` moduli orqali operatsion tizim resurslarini boshqarishni va ulanishlar arxitekturasini o'rganamiz.

1. **Server Hayotiy Sikli:** Tizim darajasidagi funksiyalar tahlili (`bind` , `listen` , `accept`).
 2. **Klient Hayotiy Sikli:** Faol ulanish algoritmi (`connect`).
 3. **Ma'lumot Almashish Mexanikasi:** Oqimli ma'lumotlar, kodlash (*UTF* — 8) va segmentatsiya.
-

2.1. Server Hayotiy Sikli (The Server Lifecycle)

Server — bu tarmoqda passiv holatda so'rov kutuvchi sub'ekt. Uning ishlashi uchun operatsion tizim darajasida quyidagi to'rtta fundamental qadam bajarilishi shart:

A. Soket yaratish (`socket.socket`)

Dastur yadrodan (kernel) aloqa uchun **fayl deskriptori** ajratishni so'raydi.

- **AF_INET:** IPv4 protokoli.
- **SOCK_STREAM:** TCP (oqimli) protokoli.

B. Manzilga bog'lash (`bind`)

Bu jarayonda soketga aniq bir **IP-manzil** va **Port** biriktiriladi.

Kiberxavfsizlik eslatmasi: Agar port 0 — 1023 oralig'ida bo'lsa, operatsion tizim administrator (root) huquqini talab qiladi. Shuning uchun biz odatda 1024+ portlardan foydalanamiz.

C. Tinglash rejimi (`listen`)

Soket "passiv" holatga o'tadi. `backlog` parametri navbatda qancha mijoz ulanishini kutishi mumkinligini belgilaydi. Agar navbat to'lib ketsa, yangi mijozlarga `ConnectionRefused` xatosi qaytadi.

D. Ulanishni qabul qilish (`accept`)

Bu **Blocking** (to'xtatib turuvchi) funksiya. Dastur shu qatorda mijoz kelguncha "muzlab" qoladi. Mijoz ulanishi bilan `accept()` ikkita ob'ekt qaytaradi:

1. **Yangi soket ob'ekti (conn):** Aynan shu mijoz bilan gaplashish uchun ochilgan alohida kanal.
 2. **Manzil (addr):** Mijozning IP va Porti.
-

2.2. Klient Hayotiy Sikli (The Client Lifecycle)

Klient faol sub'ekt bo'lib, uning asosiy vazifasi serverning "eshigini taqillatish" hisoblanadi.

- `connect((ip, port))` : Bu funksiya chaqirilishi bilan operatsion tizim darajasida **TCP Three-Way Handshake** boshlanadi. Agar server `listen` holatida bo'lmasa, ulanish darhol rad etiladi.

2.3. Ma'lumot Almashish va Segmentatsiya

TCP ma'lumotni "xabar" sifatida emas, balki **"baytlar oqimi"** sifatida ko'radi.

- `sendall(data)` : Python-dagi eng xavfsiz metod. Agar ma'lumot katta bo'lsa, u barcha baytlar jo'natilmaguncha uzatishni davom ettiradi.
- `recv(buffer_size)` : Bufer hajmi (masalan, 1024 bayt) juda muhim. Agar kelayotgan ma'lumot buferdan katta bo'lsa, u qismlarga bo'linib qoladi.

Marshaling (Kodlash):

$$Bytes = String.encode('utf - 8')$$

Dasturiy darajada biz faqat baytlar bilan ishlaymiz, shuning uchun har bir satr kodlanishi shart.

2.4. Amaliy Realizatsiya (Laboratoriya ishi)

Nazariyani amaliyotda sinash uchun biz eng sodda va ortiqcha "shovqin"lardan xoli bo'lgan kodni tahlil qilamiz. Bu kod soketlarning ishlash mexanizmini tushunish uchun poydevor bo'ladi.

Socket modulining asosiy metodlari

Python `socket` moduli tarmoq ulanishlarini boshqarish uchun quyidagi asosiy metodlardan foydalanadi[cite: 9]:

Metod	Vazifasi
<code>socket()</code>	Yangi soket yaratadi. <code>socket.AF_INET</code> (IPv4) va <code>socket.SOCK_STREAM</code> (TCP) kabi parametrlar bilan ishlaydi.
<code>bind(address)</code>	Soketni muayyan manzil (IP manzil va port) bilan bog'laydi.
<code>listen(backlog)</code>	Soketni tinglash (yangi ulanishlarni kutish) holatiga o'tkazadi.
<code>accept()</code>	Yangi ulanishni qabul qiladi va yangi soket (<code>conn</code>) bilan mijoz manzilini (<code>addr</code>) qaytaradi.
<code>connect(address)</code>	Klient soketini serverga bog'laydi (IP manzil va portga).
<code>send(data)</code>	Soket orqali ma'lumot yuboradi (qisman yuborilishi mumkin).
<code>sendall(data)</code>	Ma'lumotni to'liq yuboradi (yuborilmagan qismlar bo'lsa, davom etadi).
<code>recv(bufsize)</code>	Soketdan ma'lumot qabul qiladi. <code>bufsize</code> – maksimal qabul qilish hajmi.
<code>close()</code>	Soketni yopadi va resurslarni bo'shatadi.
<code>settimeout(value)</code>	Soketning kutish vaqtini belgilaydi (sekundlarda).

Server yaratish metodlari

Server yaratish jarayoni quyidagi metodlarni o'z ichiga oladi:

- **.socket()** : Yangi soket yaratadi. Bu metod server yoki klient uchun tarmoq ulanishi yaratishda asosiy vosita hisoblanadi.
- **.bind()** : Server soketini muayyan IP manzil va port bilan bog'laydi.
- **.listen()** : Serverni ulanishlarni kutish holatiga o'tkazadi. **backlog** parametri yangi ulanishlar uchun navbat uzunligini belgilaydi.
- **.accept()** : Server yangi ulanishni qabul qiladi va u bilan ishlash uchun yangi soket (**conn**) va mijoz manzilini (**addr**) qaytaradi.

Server kodi:

```
import socket

# 1) Soket yaratish
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2) Soketni localhost va port 5000 ga bog'lash
server_socket.bind(("127.0.0.1", 5000))

# 3) Serverni ulanishlarni kutish holatiga o'tkazish
server_socket.listen(5)
print("Server ulanishlarni kutmoqda...")

# 4) Yangi ulanishni qabul qilish
conn, addr = server_socket.accept()
print(f"Yangi mijoz ulanishi: {addr}")

# Mijozdan ma'lumot olish va qaytarish
while True:
    data = conn.recv(1024)
    if not data:
        break
    print(f"Mijozdan keldi: {data.decode('utf-8')}")
    conn.sendall(data) # Ma'lumotni qayta yuborish (echo)

conn.close()
```

Klient yaratish metodlari

Klient server bilan muloqotda quyidagi metodlarni qo'llaydi:

- **.connect()** : Klient soketini serverga bog'lash (IP manzil va portga) uchun ishlatiladi.
- **.sendall()** : Ma'lumotning to'liq yuborilishini ta'minlash uchun ishlatiladi.
- **.recv()** : Klient soketi orqali serverdan ma'lumot qabul qiladi.
- **.close()** : Klient soketini yopish, tarmoq resurslarini bo'shatish uchun ishlatiladi.

Klient kodi:

```
import socket

# 1) Soket yaratish
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2) Serverga ulanish
client_socket.connect(("127.0.0.1", 5000))

# 3) Ma'lumot yuborish
```

```
client_socket.sendall(b"Salom, server!")
```

```
# 4) Serverdan javob olish
```

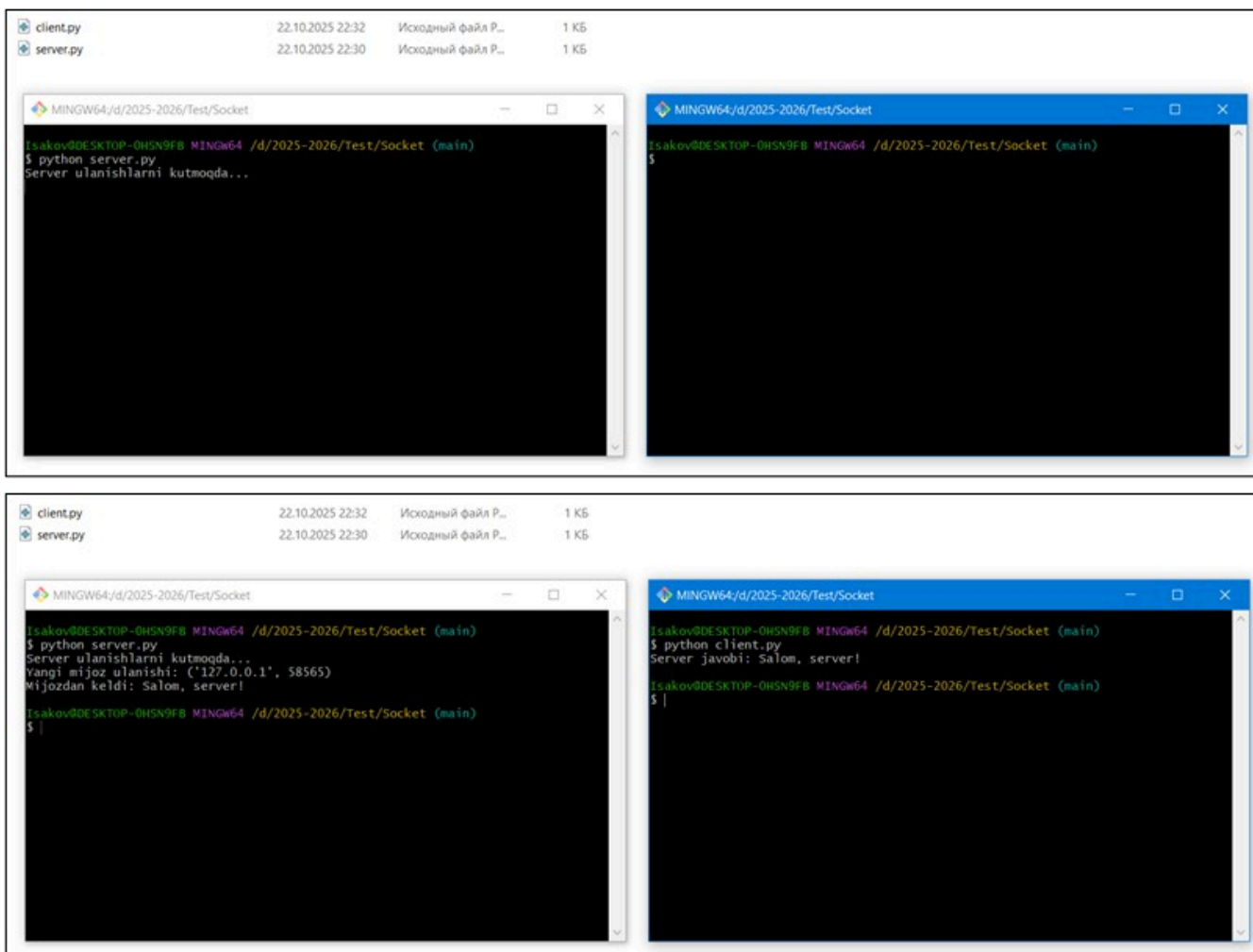
```
response = client_socket.recv(1024)
```

```
print(f"Server javobi: {response.decode('utf-8')}")
```

```
# 5) Soketni yopish
```

```
client_socket.close()
```

Klient va Server muloqotini testlash



Tarmoq muloqoti jarayonida server va klient terminallari orqali ma'lumot almashishni kuzatish mumkin:

Natija tahlili:

- Server 5000-portda tinglaydi, ammo klient har safar ulanish uchun tizim tomonidan tasodifiy portni tanlaydi.
- Server terminalida ulanish manzili va mijoz xabari ko'rinadi.
- Klient terminalida serverdan qaytgan javob (echo) aks etadi.

Xulosa:

- **Klient:** Ma'lumot yuborish uchun `.send()` va javob olish uchun `.recv()` metodlarini ishlatadi.
- **Server:** Ma'lumot qabul qilish uchun `.recv()` va javob yuborish uchun `.send()` metodlarini ishlatadi.
- Har doim aloqa yakunida `.close()` metodini chaqirish resurslarni bo'shatish uchun muhimdir.

III. Murakkab Tarmoq Ssenariylari va Xatoliklar Bilan Ishlash

Ushbu bo'limda biz tarmoq dasturlarining barqarorligini (Robustness) ta'minlash, ma'lumotlarni strukturalangan holda uzatish va xavfsizlik tahdidlaridan himoyalaniшни o'rganamiz.

3.1. Blocking vs Non-blocking Sockets: Dastur qotib qolishining oldini olish

Soketlar sukut bo'yicha **Blocking** (to'xtatib turuvchi) rejimida ishlaydi. Bu degani, `accept()` yoki `recv()` metodlari chaqirilganda, operatsiya tugallanmaguncha yoki ma'lumot kelmaguncha dasturning butun ish oqimi to'xtab qoladi.

- **Blocking Sockets:** Dastur ma'lumot kelishini "kutadi". Agar tarmoqda muammo bo'lsa yoki qarshi tomon xabar yubormasa, dastur javob bermay qoladi (freeze).
 - **Non-blocking Sockets:** `s.setblocking(False)` metodi orqali yoqiladi. Bunda metodlar kutib turmaydi; agar ma'lumot tayyor bo'lmasa, darhol xatolik qaytaradi. Bu ko'p sonli ulanishlarni bitta oqimda boshqarish imkonini beradi.
-

3.2. Timeouts va Reconnect: Tizim barqarorligi

Tarmoq ulanishi istalgan vaqtda uzilishi mumkin. Dastur bu holatda "cheksiz kutish" rejimiga tushib qolmasligi uchun **Timeout** mexanizmi kerak.

- `settimeout(value)` : Soket kutish vaqtini (masalan, 5 sekund) belgilaydi. Agar belgilangan vaqt ichida javob kelmasa, `socket.timeout` xatoligi yuz beradi.
- **Reconnect (Qayta ulanish):** Agar `connect()` xato bersa, `while` sikli ichida ma'lum vaqt oralig'ida qayta urinish algoritmini tuzish lozim.

```
import socket
```

```
s = socket.socket()
s.settimeout(5.0) # 5 sekund kutadi
```

```
try:
    s.connect(('127.0.0.1', 5000))
except socket.timeout:
    print("Xato: Ulanish vaqti tugadi (Timeout)!")
except ConnectionRefusedError:
    print("Xato: Server faol emas!")
finally:
    s.close() # Resursni bo'shatish
```

3.3. JSON Data Exchange: Murakkab ma'lumotlarni uzatish

Oddiy matnlardan ko'ra, lug'atlar (dict) va ro'yxatlar (list) bilan ishlash qulayroq. Buning uchun **JSON (JavaScript Object Notation)** formati ishlatiladi.

Jarayon:

1. Python obyektini (lug'at) JSON satriga aylantirish: `json.dumps(data)` .
 2. Satrni baytga kodlash: `.encode('utf-8')` .
 3. Tarmoqdan kelgan baytni avval dekodlash, so'ng yana Python obyektiga qaytarish:
`json.loads(decoded_data)` .
-

3.4. Kiberxavfsizlik: "Denial of Service" (DoS) hujumlari

DoS hujumi — bu serverni juda ko'p so'rovlar bilan "bo'g'ib qo'yish" va uning qonuniy foydalanuvchilarga xizmat ko'rsatishini to'xtatishdir.

- **Soket darajasidagi xavf:** Agar serverda `listen(backlog)` navbati juda katta bo'lsa va ulanishlar yopilmasa, tizim xotirasi to'lib ketadi.
 - **Himoyalaniish metodlari:**
 1. **Rate Limiting:** Bitta IP-manzildan ulanishlar sonini cheklash.
 2. **Timeout o'rnatish:** Faol bo'lmagan (idle) klientlarni avtomatik uzib qo'yish.
 3. **Validation:** Kelayotgan ma'lumot hajmini `recv(bufsize)` orqali qat'iy nazorat qilish (masalan, 1024 baytdan oshirmaslik).
-

In []:

IV. Ko'p Oqimli (Multi-threading) va Asinxron Tizimlar

Real vaqtdagi chat dasturlari uchun server bir vaqtning o'zida yuzlab foydalanuvchilarga xizmat ko'rsatishi shart. Ushbu bo'limda parallelizm asoslarini o'rganamiz.

4.1. Threading moduli: Bir vaqtning o'zida bir nechta mijozni qabul qilish

Oldingi darslardagi serverlarimiz "Iterative" (ketma-ket) edi: bitta mijoz ulanib, ishini tugatmaguncha keyingisi kutib turardi. **Threading** (oqimlar) yordamida har bir yangi ulanish uchun alohida "ishchi" (thread) ajratamiz.

- **Asosiy oqim (Main Thread):** Yangi mijozlarni kutadi (`accept()`).
- **Ishchi oqim (Worker Thread):** Ulanish sodir bo'lgach, aynan shu mijoz bilan muloqot qiladi.

Amaliy namuna:

```
import socket
import threading

def handle_client(conn, addr):
    print(f"[NEW CONNECTION] {addr} ulandi.")
    connected = True
    while connected:
        data = conn.recv(1024)
        if not data: break
        print(f"[{addr}] {data.decode('utf-8')}")
        conn.sendall("Xabar yetib bordi".encode('utf-8'))
    conn.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('127.0.0.1', 5050))
server.listen()

print("[STARTING] Server ishga tushdi...")
while True:
    conn, addr = server.accept()
    # Har bir mijoz uchun alohida oqim yaratish
    thread = threading.Thread(target=handle_client, args=(conn, addr))
    thread.start()
    print(f"[ACTIVE CONNECTIONS] {threading.active_count() - 1}")
```

4.2. Race Conditions va Lock mexanizmi

Ko'p oqimli tizimlarda bitta muammo bor: agar ikkita oqim bir vaqtning o'zida bitta umumiy o'zgaruvchini (masalan, umumiy chat tarixi yoki foydalanuvchilar ro'yxati) o'zgartirmoqchi bo'lsa, ma'lumotlar buziladi. Bu **Race Condition** (poyga holati) deyiladi.

- **Lock (Qulflash):** Bir vaqtning o'zida faqat bitta oqim ma'lumotni o'zgartirishi uchun uni "qulflab" qo'yish.

```
import threading

lock = threading.Lock()
chat_history = []

def update_history(message):
    with lock: # Faqat bitta thread kiradi
        chat_history.append(message)
```

4.3. Asyncio kutubxonasi: High-load tizimlar

Threading resurs talab qiladi (har bir oqim RAM egallaydi). Minglab ulanishlar uchun **Asyncio** (Asinxron kirish-chiqish) ko'proq mos keladi.

- **Asinxronlik:** Dastur ma'lumot kelishini kutib o'tirmaydi, boshqa ishlarni bajarib turadi. Ma'lumot tayyor bo'lganda unga qaytadi.
 - **Event Loop:** Barcha hodisalarni (events) bitta oqimda boshqaruvchi markaz.
-

4.4. Kiberxavfsizlik: Thread Exhaustion Attack

Agar serveringizda ulanishlar soni cheklanmagan bo'lsa, buzg'unchi minglab ulanishlarni ochib, server xotirasini (RAM) to'ldirib yuborishi mumkin.

Himoya choralari:

1. **Thread Pool:** Bir vaqtning o'zida ochiladigan oqimlar sonini (masalan, maksimal 100 ta) cheklash.
2. **Keep-alive Timeout:** Ma'lumot yubormayotgan oqimlarni majburiy yopish.

In []:

V. Grafik Interfeys (PyQt5) va Chat Loyihasi

Ushbu bo'limda biz tarmoq dasturlash bilimlarimizni zamonaviy grafik foydalanuvchi interfeysi (GUI) bilan birlashtiramiz.

5.1. PyQt5 GUI Dizayni: Signals & Slots

Tarmoq ilovalari uchun interfeys yaratishda PyQt5 ning **Signal-Slot** mexanizmi markaziy o'rin tutadi. Bu tizim hodisalarga (masalan, tugma bosilishi yoki xabar kelishi) asoslangan dasturlashni ta'minlaydi.

- **Chat oynasi:** Xabarlarni ko'rsatish uchun `QTextEdit` va yozish uchun `QLineEdit` vidjetlaridan foydalaniladi.
 - **Signallar:** Tarmoqdan yangi xabar kelganda, maxsus signal (`pyqtSignal`) orqali interfeys yangilanadi.
 - **Kiberxavfsizlik:** Login formasida parollarni ochiq ko'rinishda saqlamaslik va kiritish maydonida `EchoMode` ni `Password` qilib sozlash zarur.
-

5.2. Network-Worker Pattern: Oqimlarni ajratish

Tarmoq dasturlashda eng katta xato — tarmoq kodini (masalan, `recv()`) asosiy GUI oqimida (Main Thread) ishlatishdir.

- **Muammo:** `recv()` metodining tabiatiga ko'ra, u ma'lumot kelguncha "blocking" (to'xtatib turuvchi) rejimda bo'ladi. Agar u asosiy oqimda bo'lsa, interfeys qotib qoladi (Not Responding).
 - **Yechim:** Tarmoq kodi alohida **Worker Thread** (`QThread`) ichida ishlashi kerak.
-

5.3. Real-time Chat: Broadcasting (Xabarlarni tarqatish)

Haqiqiy chat dasturi uchun serverga **Broadcasting** algoritmi kerak. Ya'ni, bir mijozdan kelgan xabar barcha faol ulanishlarga (klientlarga) yuborilishi shart.

Broadcasting Algoritmi:

1. Barcha faol mijoz soketlarini bitta ro'yxatda (`list`) saqlash.
 2. Yangi xabar kelganda, tsikl orqali ro'yxatdagi har bir soketga ma'lumotni yuborish.
 3. **Xavfsizlik:** Agar biror mijoz ulanishi uzilsa (`ConnectionReset`), uni darhol ro'yxatdan o'chirish lozim, aks holda server "o'lik" soketlarga ma'lumot yuborishga urinib, qulab tushishi mumkin.
-

Amaliy Kod: PyQt5 Chat Klienti (Soddalashtirilgan Model)

Ushbu kodni Jupyter Notebook-da ishlatsangiz, alohida oyna ochiladi. (Kompyuteringizda `pip install PyQt5` o'rnatilgan bo'lishi shart).

```
from PyQt5.QtWidgets import QApplication, QMainWindow, QTextEdit, QLineEdit,
QVBoxLayout, QWidget
from PyQt5.QtCore import QThread, pyqtSignal
```



```

import socket

# Tarmoq ishlarini bajaruvchi alohida oqim
class ReceiverThread(QThread):
    message_received = pyqtSignal(str)

    def run(self):
        client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        client.connect(('127.0.0.1', 5050))
        while True:
            try:
                data = client.recv(1024).decode('utf-8')
                if data:
                    self.message_received.emit(data)
            except:
                break

class ChatGUI(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Real-time Chat")
        self.display = QTextEdit(self)
        self.input = QLineEdit(self)

        layout = QVBoxLayout()
        layout.addWidget(self.display)
        layout.addWidget(self.input)

        container = QWidget()
        container.setLayout(layout)
        self.setCentralWidget(container)

        # Worker Threadni ishga tushirish
        self.thread = ReceiverThread()
        self.thread.message_received.connect(self.display.append)
        self.thread.start()

# Dasturni ishga tushirish qismi
# app = QApplication([])
# gui = ChatGUI()
# gui.show()
# app.exec_()

```

In []:

VI. Kiberxavfsizlik: Ma'lumotlarni Shifrlash (End-to-End)

Tarmoq dasturlashda xavfsizlik — bu ixtiyoriy tanlov emas, balki majburiyatdir. Ushbu bo'limda ma'lumotlarni begona ko'zlardan himoya qilish usullarini ko'rib chiqamiz.

6.1. Simmetrik shifrlash: AES algoritmi

AES (Advanced Encryption Standard) — hozirgi kunda dunyoda eng keng tarqalgan va xavfsiz simmetrik shifrlash algoritmi hisoblanadi. "Simmetrik" deyilishiga sabab, ma'lumotni shifrlash va uni qayta o'qish (deshifrlash) uchun **bitta umumiy kalit** ishlatiladi.

- **Ishlash prinsipi:** Matnli xabar kalit yordamida o'qib bo'lmaydigan baytlar ketma-ketligiga aylantiriladi.
 - **Kiberxavfsizlik: E2EE (End-to-End Encryption):** Xabarlar klient qurilmasida shifrlanadi va faqat qabul qiluvchi qurilmasida ochiladi. Server xabarni uzatadi, lekin kalitga ega bo'lmagani uchun uni o'qiy olmaydi.
 - **Python realizatsiyasi:** Buning uchun odatda `cryptography` kutubxonasining `Fernet` (AES-128 asosida) moduli ishlatiladi.
-

6.2. SSL/TLS asoslari: Sertifikatlar va Xavfsiz kanal

Agar AES ma'lumotning o'zini himoya qilsa, **SSL/TLS (Secure Sockets Layer / Transport Layer Security)** butun aloqa kanalini himoyalaydi. Bu HTTPS protokoli asosi hisoblanadi.

- **Handshake jarayoni:** TCP ulanish o'rnatilgandan so'ng, "TLS Handshake" boshlanadi. Bunda server o'zining **Raqamli sertifikatini** (Digital Certificate) taqdim etadi.
 - **Asimmetrik shifrlash:** Ulanish boshida ochiq (public) va yopiq (private) kalitlar juftligi ishlatiladi. Bu xuddi qulf va kalitdek gap: qulf hamma uchun ochiq, lekin uni faqat egasidagi kalit ochadi.
 - **Ma'lumotlar butunligi (Integrity):** TLS ma'lumot yo'lda o'zgartirilmaganligini tekshirish uchun "Message Authentication Code" (MAC) dan foydalanadi.
-

Amaliy misol: Xabarni AES bilan shifrlash

Chat dasturingizga shifrlashni qo'shish uchun `cryptography` paketidan foydalanamiz:

```
from cryptography.fernet import Fernet
```

```
# 1. Kalit yaratish (Buni faqat bir marta qilib, xavfsiz saqlash kerak)
```

```
key = Fernet.generate_key()
```

```
cipher_suite = Fernet(key)
```

```
# 2. Xabarni shifrlash (Yuborishdan oldin)
```

```
message = "Kiberxavfsizlik darsi juda qiziq!".encode('utf-8')
```

```
cipher_text = cipher_suite.encrypt(message)
```

```
print(f"Shifrlangan xabar (tarmoqqa ketadi): {cipher_text}")
```

```
# 3. Xabarni ochish (Qabul qilingandan so'ng)
plain_text = cipher_suite.decrypt(cipher_text)
print(f"Asl xabar: {plain_text.decode('utf-8')}")
```

Kiberxavfsizlik Tahlili: Man-in-the-Middle (MitM)

Agar siz shifrlashdan foydalanmasangiz, tarmoqdagi istalgan uchinchi shaxs (masalan, ochiq Wi-Fi egasi) sizning paketlaringizni tutib olib, chatdagi yozishmalaringizni o'qishi mumkin. **TLS** va **AES** birgalikda ishlatilganda, tajovuzkor paketni tutib olsa ham, uning ichidagi ma'lumotni o'qiy olmaydi, chunki u shunchaki "chalkash baytlar" bo'lib ko'rinadi.

In []:

VII. Deployment: Loyihani Mustaqil Faylga Aylantirish

O'quv qo'llanmamizning yakuniy bosqichida biz yaratgan Python kodimizni oddiy foydalanuvchi ishga tushira oladigan (.exe) holatiga keltiramiz.

7.1. PyInstaller: .exe fayl yaratish va paketlash

Dasturni boshqa kompyuterga yuborganingizda, u yerda Python o'rnatilgan bo'lmasligi yoki kerakli kutubxonalar (`PyQt5` , `cryptography`) yetishmasligi mumkin. **PyInstaller** barcha bog'liqliklarni bitta fayl ichiga toplaydi.

- **Vazifasi:** Python skriptini operatsion tizim tushunadigan ikkilik (binary) faylga o'tkazish.
 - **Asosiy komandalar:**
 - `pip install pyinstaller` — dasturni o'rnatish.
 - `pyinstaller --onefile --windowed main.py`
 - `--onefile` : Hamma narsani bitta `.exe` faylga jamlaydi.
 - `--windowed` : Dastur ishga tushganda orqada qora konsol oynasi chiqishini oldini oladi (GUI dasturlar uchun muhim).
-

7.2. Cross-platform Testing: Windows va Linux

Python "cross-platform" tildir, ya'ni uning kodi hamma joyda ishlaydi. Biroq, `.exe` fayllar faqat Windows-da ishlaydi.

- **Windows uchun:** Windows kompyuterida PyInstaller-ni ishga tushiring.
 - **Linux uchun:** Linux tizimida (masalan, Ubuntu) PyInstaller-dan foydalaning, u sizga `ELF` formatidagi (Linux uchun dastur) fayl beradi.
 - **Testing (Sinov):** Dasturni boshqa kompyuterda ishga tushirishdan oldin, tarmoq ulanishlari va xavfsizlik devori (Firewall) portlarni to'sib qo'ymayotganini tekshirish kerak.
-

7.3. Kiberxavfsizlik: Antiviruslar va "False Positive"

Yangi yaratilgan `.exe` fayllar ko'pincha antiviruslar tomonidan "shubhali" deb topiladi.

- **Sababi:** Sizning dasturingiz tarmoqqa ulanadi (socket) va ma'lumotlarni shifrlaydi (AES). Bu xatti-harakatlar ba'zi viruslarga o'xshab ketishi mumkin.
 - **Yechim:** Professional darajada dasturga "Raqamli imzo" (Code Signing) qo'shish kerak. O'quv loyihalarida esa shunchaki antivirusga "istisno" (exclusion) sifatida qo'shish kifoya.
-

Yakuniy Xulosa

Siz ushbu kurs davomida fundamental tarmoq bilimlaridan boshlab, real vaqtdagi shifrlangan chat tizimini yaratishgacha bo'lgan yo'lni bosib o'tdik